ARQUITETURA DE

SISTEMAS

E-COMMERCE · PARTE 4: SAGA E RESILIÊNCIA

Prof. Augusto Gehrke

CONTEÚDO

1. Transações que atravessam serviços
2. Saga: passos locais e compensações
3. Coreografia × orquestração; as anomalias
4. Falhas: timeout, retry e idempotência
5. Circuit breaker e fallback
6. Saga e resiliência na prática (docker-class)
7. No projeto: o pagamento entra na saga

HOJE: QUANDO DÁ ERRADO

O pedido já anda sozinho até reservar o estoque. Falta cobrar. E cobrar envolve um banco que não é nosso, que demora, que cai e que recusa.

1. **Saga:** como desfazer um passo que já foi gravado em outro serviço
2. **Resiliência:** timeout, retry e circuit breaker contra um banco instável
3. **Ver ao vivo** na docker-class: compensação, estorno e o circuito abrindo
4. **No projeto:** pagamentos, banco-falso e as compensações

CONTINUA EM DUPLA, no mesmo repositório.

ONDE PARAMOS



Até a aula passada: o checkout publica pedido.criado, o catálogo reserva e o pedido fica em AGUARDANDO_PAGAMENTO.

Hoje entra o pagamento. E com ele, a pergunta que faltava:

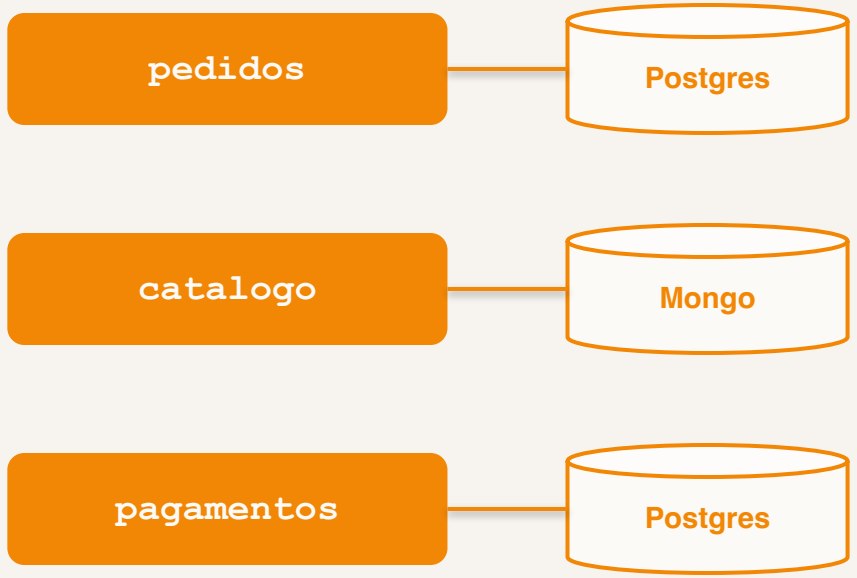
O estoque já foi reservado e o cartão foi recusado. Quem devolve o estoque?

O cliente cancelou depois de pago. Quem devolve o dinheiro? E o estoque?

O banco do cartão não responde. Esperamos quanto? Tentamos de novo? E se ele cobrou e só não respondeu?

UMA COMPRA, TRÊS BANCOS

```sql
-- no monolito: um banco só
BEGIN;
  INSERT INTO pedidos ...;
  UPDATE produtos SET estoque = estoque - 1;
  INSERT INTO pagamentos ...;
COMMIT;   -- ou ROLLBACK: desfaz tudo
```



Em microserviços, cada passo grava num banco diferente, em outro serviço. Não existe um ROLLBACK que desfaça os três.

Quando o pagamento falha, a reserva do estoque já foi gravada e confirmada no banco do catálogo, há segundos.

A propriedade que se perde é o A de ACID: atomicidade, o "tudo ou nada" entre serviços.

E UMA TRANSAÇÃO DISTRIBUÍDA?

Existe: o commit em duas fases (2PC). Um coordenador pergunta a todos "pode confirmar?" e só confirma se todos disserem sim.

POR QUE QUASE NINGUÉM USA

Todos precisam estar no ar ao mesmo tempo: um lento trava todos.

Os registros ficam bloqueados enquanto o coordenador decide.

Kafka, Mongo e um banco de cartão externo não participam de um 2PC.

O QUE SE ESCOLHE NO LUGAR

Cada serviço confirma a sua parte na hora, no próprio banco.

Se algo der errado depois, desfaz com uma nova ação.

Disponibilidade agora, consistência daqui a pouco: a saga.

O preço: por alguns segundos, o sistema mostra estados intermediários (consistência eventual, da aula passada).

SAGA

Uma operação longa vira uma sequência de transações locais, uma em cada serviço.

Cada passo tem uma compensação: uma ação que desfaz o seu efeito, caso um passo seguinte falhe.

A ideia é de 1987 (Garcia-Molina e Salem, para transações longas em bancos de dados) e voltou com os microserviços.

RESUMO: no monolito, o banco desfazia tudo; aqui, cada serviço desfaz a própria parte.

PASSOS E COMPENSAÇÕES

Passo (transação local)	Serviço	Compensação	Disparada por
Criar o pedido	pedidos	marcar CANCELADO	estoque.insuficiente, pagamento.recusado
Reservar o estoque	catalogo	devolver o estoque	pagamento.recusado, pedido.cancelado
Cobrar o cartão	pagamentos	estornar a cobrança	pedido.cancelado
Avisar o cliente	notificacoes	mandar outro e-mail	qualquer cancelamento

Cada serviço conhece só a própria compensação. O catálogo não sabe por que o pedido falhou; ele escuta pagamento.recusado e devolve o estoque.

A saga para onde falhou: sem estoque, o pagamento nem é chamado, e não há nada a estornar.

Compensação também é evento: a devolução e o estorno são publicados, e quem precisa saber reage.

COMPENSAR NÃO É APAGAR

ROLLBACK

Faz de conta que nunca aconteceu.

Ninguém viu o estado intermediário.

Só existe dentro de um banco, numa transação.

COMPENSAÇÃO

Uma ação nova que anula o efeito: estorno, devolução, e-mail de cancelamento.

O histórico fica: o pedido existiu e foi cancelado.

Outros podem ter visto o estado intermediário.

Como no mundo real: a loja não apaga a compra do extrato do cartão; ela lança um estorno. O cliente vê as duas linhas.

Algumas coisas não se compensam: e-mail enviado, mercadoria entregue. O que é irreversível vai o mais para o fim possível.

UMA SAGA QUE FALHA NO PAGAMENTO



Lendo da esquerda para a direita: o pedido nasce, o catálogo reserva, o banco recusa. O pagamento publica pagamento.recusado.

Na última coluna, em amarelo, as compensações: o catálogo devolve o estoque e o pedido vira CANCELADO. Ninguém mandou: cada um escutou.

É o passo 3 do roteiro da docker-class: o histórico do pedido mostra cada mudança e o evento que a causou.

A ORDEM DOS PASSOS IMPORTA

COMPENSÁVEIS

Podem ser desfeitos: reservar estoque, criar pedido. Vão primeiro.

PONTO SEM VOLTA

Depois dele, a saga só segue em frente. Ex.: despachar a mercadoria.

SÓ PARA A FRENTE

Não podem falhar de vez: tentam até conseguir. Ex.: avisar o cliente, gerar a nota.

Regra: o que é fácil de desfazer vem antes; o que é difícil ou impossível, por último.

Por isso o estoque é reservado antes de cobrar: devolver estoque é barato; estornar custa taxa, e o cliente vê no extrato.

E por isso o e-mail "pedido confirmado" só sai depois do pagamento aprovado. Antes, só "recebemos o seu pedido".

QUEM CONDUZ A SAGA?

COREOGRAFIA (o nosso projeto)



Cada serviço escuta eventos e publica os seus.

Ninguém conhece o fluxo inteiro.

Serviço novo: só começa a escutar.

ORQUESTRAÇÃO



Um coordenador manda comandos: "reserve", "cobre", "devolva".

O fluxo inteiro está num lugar só.

Ferramentas: Temporal, Camunda, AWS Step Functions.

	Coreografia	Orquestração
Bom quando	poucos passos, fluxo simples	muitos passos, regras e prazos
Cuidado com	fluxo espalhado, difícil de ver	orquestrador vira o centro de tudo

SAGA NÃO TEM ISOLAMENTO

ACID sem o I: enquanto a saga anda, os outros enxergam os passos já feitos, mesmo que eles ainda possam ser desfeitos.

Anomalia	No e-commerce
Leitura de algo que vai sumir	outro cliente vê "esgotado" enquanto a reserva de um pedido que vai ser recusado ainda existe
Decisão com dado provisório	um relatório soma como venda um pedido em AGUARDANDO_PAGAMENTO
Eventos cruzados	o cliente cancela enquanto o banco aprova: chegam pedido.cancelado e pagamento.aprovado

O status funciona como cadeado: AGUARDANDO_* diz "isto ainda pode mudar". Relatórios e telas tratam esses estados à parte.

Mudança só a partir do estado esperado: PAGO só a partir de AGUARDANDO_PAGAMENTO. Um pedido CANCELADO não volta a PAGO porque o banco aprovou atrasado.

O CLIENTE CANCELA NO MEIO DA COBRANÇA

O cancelamento chegou...	pagamentos faz	Resultado
antes da cobrança começar	grava CANCELADO no lugar do pagamento	o estoque.reservado que chegar depois é ignorado; o banco nem é chamado
depois da cobrança aprovada	estorna no banco e publica pagamento.estornado	o pedido continua CANCELADO; o estoque volta

```
// pagamentos, ao receber pedido.cancelado
INSERT INTO pagamentos (pedido_id, status) VALUES ($1, 'CANCELADO') ON CONFLICT DO NOTHING
// gravou? não tinha cobrado: o pedido_id ocupado impede a cobrança futura
// não gravou? já existe: se está APROVADO, estorna
```

Os dois finais estão certos: nenhum deixa o cliente pagando por um pedido cancelado. É o passo 5 do roteiro.

E SE A COMPENSAÇÃO FALHAR?

- ✓ **Compensação precisa dar certo:** não dá para "compensar a compensação". Ela tenta de novo até conseguir.

- ✓ **Por isso, idempotente:** devolver o estoque duas vezes não pode acontecer. Filtro de estado: só devolve se a reserva ainda está RESERVADA.

- ✓ **Sem prazo infinito:** depois de N tentativas, vai para um registro de pendências (ou um tópico de erros) e alguém resolve.

- ✓ **O estorno não passa pelo circuit breaker** na docker-class: devolver dinheiro é importante demais para ser barrado.

Na vida real: toda empresa que cobra tem uma rotina de conciliação: compara o que o banco diz que cobrou com o que o sistema registrou, e corrige a diferença.

RESILIÊNCIA

Num sistema distribuído, a pergunta não é SE algo vai falhar, é QUANDO.

Resiliência é continuar funcionando, talvez pior, enquanto uma parte está fora: sem derrubar o resto junto.

Hoje: o banco do cartão, um sistema que não é nosso.

TRÊS PADRÕES: timeout, retry e circuit breaker.

O QUE TODO MUNDO SUPÕE (E É FALSO)

As falácias da computação distribuída (Peter Deutsch e colegas, Sun Microsystems, anos 90):

A rede é confiável	**A latência é zero**	**A banda é infinita**	**A rede é segura**
A topologia não muda	**Existe um só administrador**	**O transporte não custa nada**	**A rede é homogênea**

As duas primeiras são o assunto de hoje. Toda chamada pela rede pode:

- não chegar;
- chegar, ser processada e a resposta não voltar;
- demorar 10 vezes mais que o normal;
- dar erro uma vez e funcionar na seguinte.

LENTO É PIOR QUE FORA DO AR



Fora do ar: a conexão é recusada em milissegundos. Quem chama descobre na hora e segue.

Lento: cada requisição fica presa esperando, segurando memória, conexão e uma vaga no servidor.

A falha em cascata: o banco fica lento → os pagamentos enchem de requisições presas → param de responder → quem chama os pagamentos também fica preso.

Um serviço lento derruba os vizinhos, mesmo que eles não tenham nada de errado.

Os três padrões de hoje existem para isto: não esperar para sempre (timeout), insistir só o necessário (retry) e parar de chamar quem está no chão (circuit breaker).

TIMEOUT: NÃO ESPERAR PARA SEMPRE

```
const r = await fetch(`${BANK_URL}/cobrancas`, {
  method: 'POST', body: ...,
  signal: AbortSignal.timeout(2000),  // desiste em 2 s
});
```

Sem timeout, muitas bibliotecas esperam para sempre (ou por minutos).

Toda chamada pela rede precisa de um: HTTP, banco, cache.

Quanto? Um pouco acima do tempo das respostas lentas normais: se 99% das cobranças respondem em 800 ms, 2 s é razoável.

Curto demais: desiste de chamadas que iam dar certo.

Longo demais: prende recursos e o cliente espera.

Em cadeia: o timeout de fora precisa ser maior que o de dentro. Se o Kong desiste em 5 s, o serviço não pode esperar o banco por 10 s.

Na docker-class: 2 s para o banco. No passo 6, o banco leva 1,5 s e dá certo; no passo 7, leva 3 s e estoura.

TIMEOUT NÃO É "NÃO ACONTECEU"

```
[Banco]      cobrança 7 de R$ 60: aprovada (TX-F6A1)
[Pagamentos] tentativa 1/3 falhou: sem resposta em 2000 ms (timeout)
[Banco]      cobrança 7 REPETIDA: devolvendo a mesma (TX-F6A1), sem cobrar de novo
[Pagamentos] tentativa 2/3 falhou: ... (timeout)
[Pagamentos] tentativa 3/3 falhou: ... (timeout)
[Banco]      ESTORNO da cobrança TX-F6A1 (pedido 7, R$ 60)
```

O banco COBROU. Só a resposta não chegou a tempo. Para quem chama, timeout quer dizer "não sei o que aconteceu".

Se o retry cobrasse de novo, o cliente pagaria duas vezes. Retry só é seguro se o outro lado reconhecer a repetição (próximo slide).

Desistiu sem saber? Desfaça por segurança: o pagamento pede o estorno da referência. Se não havia cobrança, o banco responde 404 e tudo bem.

RETRY: TENTAR DE NOVO, COM CRITÉRIO

Resposta	Tentar de novo?	Por quê
timeout, conexão recusada, 503	sim	falha passageira: pode dar certo daqui a pouco
429 (muitas requisições)	sim, mais tarde	o outro lado pediu calma
400, 402, 404, 409	não	é uma resposta: insistir não muda o "não"

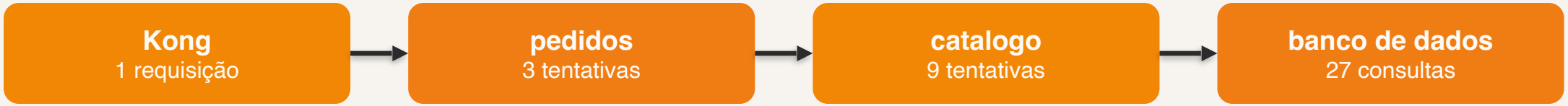
```
for (tentativa = 1; tentativa <= 3; tentativa++) {
  try { return await chamarBanco(...); }
  catch { await esperar(500 * 2 ** (tentativa - 1)); }
}  // espera 500 ms, depois 1000 ms
```

Espera crescente (backoff): dá tempo para o outro lado se recuperar.

Jitter: um pouco de aleatoriedade na espera, para mil clientes não tentarem todos no mesmo milissegundo.

Limite: poucas tentativas. 30% de falha vira ~3% com 3 tentativas (roteiro, passo 8).

CUIDADO: RETRY MULTIPLICA



Retry em todas as camadas: 3 × 3 × 3 = 27 chamadas para cada clique, justamente quando o sistema lá no fundo já está sofrendo.

O retry vira um ataque ao próprio sistema e impede que ele se recupere.

Como evitar:

- retry em uma camada só, a mais próxima de quem falha (no nosso caso, pagamentos → banco);
- poucas tentativas, espera crescente e jitter;
- circuit breaker: quando falhar muito, parar de tentar de vez.

CHAVE DE IDEMPOTÊNCIA

```
// quem chama: a mesma referência em toda tentativa
POST /cobrancas
{ "valor": 60, "referencia": "7" }

// o banco: já vi essa referência?
if (cobrancas.has(referencia))
  return a mesma resposta;  // não cobra
```

A referência é o id do pedido: o banco guarda as referências que já cobrou e devolve a mesma resposta para a repetida.

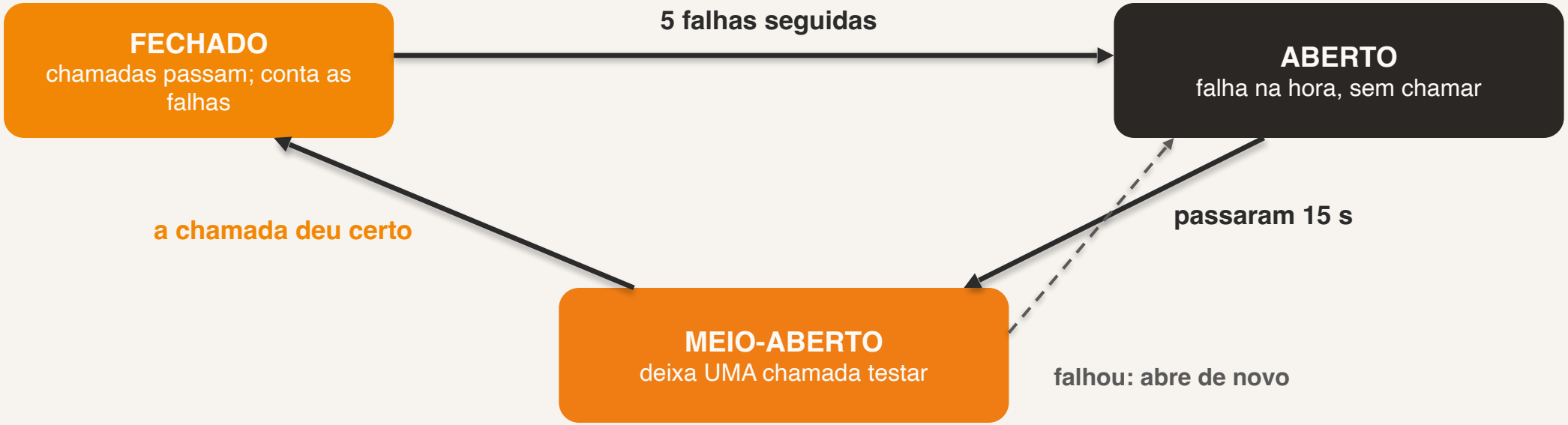
É o mesmo princípio do pedido_id UNIQUE da aula de Kafka, agora do lado de quem recebe a chamada HTTP.

No mercado: o cabeçalho Idempotency-Key, usado por Stripe, Mercado Pago e outros gateways de pagamento.

Regra: retry só em operação idempotente. Ler (GET) é idempotente por natureza. Cobrar, criar e enviar precisam de uma chave.

Sem a chave, o retry do passo 7 teria cobrado o cliente três vezes.

CIRCUIT BREAKER: O DISJUNTOR



Como o disjuntor da sua casa: quando algo dá curto, ele desarma para não queimar o resto. Aqui, para não insistir num serviço que está no chão.

Duas vantagens: quem chama descobre na hora (não fica preso esperando) e o serviço doente ganha um respiro para se recuperar.

O meio-aberto é o jeito de descobrir sozinho que o outro voltou, sem ninguém ligar o disjuntor na mão.

CIRCUIT BREAKER DO ZERO

```
const circuito = { estado: 'FECHADO', falhasSeguidas: 0, abertoEm: null };

function podeChamar() {
  if (circuito.estado === 'ABERTO' && Date.now() - circuito.abertoEm >= 15000)
    circuito.estado = 'MEIO_ABERTO';     // hora de testar
  return circuito.estado !== 'ABERTO';
}
function registrarSucesso() {
  circuito.estado = 'FECHADO'; circuito.falhasSeguidas = 0;
}
function registrarFalha() {
  circuito.falhasSeguidas++;
  if (circuito.estado === 'MEIO_ABERTO' || circuito.falhasSeguidas >= 5) {
    circuito.estado = 'ABERTO'; circuito.abertoEm = Date.now();
  }
}

// antes de cada chamada:
if (!podeChamar()) throw new CircuitoAberto();
```

Umas 25 linhas em payment-service/banco.js, na docker-class.

O que conta como falha: timeout, erro de rede e 5xx. Um 402 é o banco respondendo "não": é sucesso para o circuito.

Com o circuito aberto, o retry nem tenta: não adianta insistir.

Em produção, uma biblioteca: opossum (Node), Resilience4j (Java), Polly (C#).

NO PROJETO: OPOSSUM

```
const CircuitBreaker = require('opossum');

const disjuntor = new CircuitBreaker(chamarBanco, {
  timeout: 2500,                 // o opossum também corta
  errorThresholdPercentage: 50,  // abre com 50% de falha...
  volumeThreshold: 5,            // ...em pelo menos 5 chamadas
  resetTimeout: 15000,           // 15 s até o meio-aberto
});
disjuntor.on('open', () => console.log('CIRCUITO ABERTO'));

const resultado = await disjuntor.fire(dados);
disjuntor.opened   // true quando aberto
```

Mesma ideia, contando de outro jeito: percentual de falhas numa janela de tempo, em vez de falhas seguidas.

volumeThreshold: sem ele, a primeira chamada que falhasse já seria 100% de falha e abriria o circuito.

Eventos: open, halfOpen, close. Use para logar e para mostrar o estado no /health.

FALLBACK: E QUANDO NÃO DÁ?

Estratégia	Exemplo	Quando faz sentido
Falhar rápido	pagamento recusado: banco indisponível	o cliente pode tentar de novo
Deixar para depois	pedido em espera, cobrança tentada mais tarde	a venda vale mais que a demora
Resposta guardada	vitrine do cache com o Redis fora (X-Cache: OFF)	dados que mudam pouco
Valor padrão	frete fixo quando a transportadora não responde	melhor um valor aproximado que nada
Funcionalidade a menos	sem recomendações, a loja continua vendendo	o que não é essencial

Fallback é decisão de negócio, não de código. Cancelar o pedido porque o banco caiu é o mais simples (e o que fazemos); deixar o pedido esperando não perde a venda, mas dá mais trabalho.

A CAIXA DE FERRAMENTAS

Padrão	O que faz	Onde vimos
Timeout	desiste depois de um tempo	pagamentos → banco (2 s)
Retry com backoff	tenta de novo, esperando cada vez mais	pagamentos → banco (3×)
Circuit breaker	para de chamar quem está falhando	pagamentos → banco
Idempotência	repetir não muda o resultado	banco, pagamentos, catálogo
Fila (mensageria)	o trabalho espera o serviço voltar	Kafka, entre todos
Rate limit	limita quantas chamadas cada cliente faz	Kong, no login
Bulkhead	separa recursos: um vizinho lento não ocupa tudo	uma conexão por destino
Health check	avisa se está de pé	/health de cada serviço

Nenhum resolve sozinho. Juntos: timeout por chamada, retry por cima, circuit breaker em volta, e a saga para desfazer o que sobrar.

AO VIVO: SAGA NA DOCKER-CLASS

order, stock e payment em saga, e um banco de cartão de mentira que demora e falha quando a gente manda.

1. Caminho feliz: o histórico do pedido
2. Sem estoque: a saga para
3. Cartão recusado: o estoque volta
4. Cancelado depois de pago: estorno
5. Cancelado no meio da cobrança
6. Banco lento, dentro do timeout
7. Timeout: cobrou, não respondeu
8. 30% de falha: o retry resolve
9. Banco fora: o circuito abre e fecha
10. Estoque fora: a saga espera

ROTEIRO: `docker-class/ROTEIRO-SAGA.md`

ENXERGAR A FALHA

- ✓ **O estado do circuito no /health:** circuitoBancoExterno: aberto. Quem olha o painel sabe na hora o que está acontecendo.
- ✓ **Logs com o correlationId:** a tentativa, o estorno e a compensação do mesmo pedido aparecem juntos no grep.
- ✓ **Status e motivo gravados:** pagamento ERRO, 3 tentativas, "banco indisponível (timeout)". O suporte responde o cliente sem abrir o código.
- ✓ **Métricas que importam:** taxa de erro, tempo de resposta das mais lentas (p99), lag dos consumers.

Resiliência sem observabilidade esconde problemas: o retry resolve em silêncio até o dia em que não resolve mais.

Logue cada tentativa.

NO PROJETO: O PAGAMENTO ENTRA NA SAGA

O pedido para em AGUARDANDO_PAGAMENTO desde a aula passada.

Hoje entram o serviço de pagamentos e o banco-falso. E cada serviço aprende a desfazer a sua parte quando o pagamento é recusado ou o pedido é cancelado.

ENTRA NO COMPOSE: pagamentos, pagamentos-db e banco-falso.

A SAGA COMPLETA DO E-COMMERCE



Resultado do pagamento	O que cada serviço faz
pagamento.aprovado	pedidos → PAGO · catalogo confirma a reserva (sai do reservado) · notificacoes avisa
pagamento.recusado	pedidos → CANCELADO · catalogo DEVOLVE o estoque · notificacoes avisa
pedido.cancelado (cliente)	catalogo DEVOLVE o estoque · pagamentos ESTORNA, ou marca CANCELADO se ainda não cobrou
pagamento.estornado	notificacoes avisa o cliente do estorno

Recusa do banco e banco fora do ar dão no mesmo evento, pagamento.recusado, com motivos diferentes. A saga compensa igual.

BANCO-FALSO: O SISTEMA EXTERNO

Entrada	Resposta
POST /cobrancas, cartão final 0002	402 { motivo: "saldo insuficiente" }
POST /cobrancas, outro cartão ou pix	201 { codigo: "TX-..." }
falhaPercent (0 a 100)	chance de responder 503
latenciaMs	quanto demora para responder
POST /estornos { codigo, referencia }	201
GET / POST /config	ver e mudar latência e falhas ao vivo

Fora do Kong, com porta própria (9000): ele representa o banco do cartão, que não é nosso. Ninguém de fora deveria chamar o banco pela loja.

Sem banco de dados: é uma simulação de umas 50 linhas. Opcional: não cobrar duas vezes a mesma referência, como na docker-class.

PAGAMENTOS: A TABELA (POSTGRES)

```
CREATE TABLE pagamentos (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pedido_id        UUID NOT NULL UNIQUE,   -- idempotência
  usuario_id       UUID NOT NULL,
  valor            NUMERIC(12,2) NOT NULL,
  forma            TEXT, cartao_final TEXT,
  status           TEXT NOT NULL CHECK (status IN (
    'PENDENTE', 'APROVADO', 'RECUSADO', 'ERRO',
    'CANCELADO', 'ESTORNADO')),
  tentativas       INT NOT NULL DEFAULT 0,
  codigo_transacao TEXT,                -- o TX-... do banco
  motivo           TEXT,
  criado_em        TIMESTAMPTZ DEFAULT now()
);
```

Rotas: GET /pagamentos/:pedidoId (dono ou admin) · GET /pagamentos (admin) · GET /health com o estado do circuito.

PENDENTE: gravado antes de chamar o banco.

APROVADO / RECUSADO: o banco respondeu.

ERRO: o banco não respondeu (timeout, 503, circuito aberto).

CANCELADO: o pedido foi cancelado antes da cobrança.

ESTORNADO: cobrado e devolvido.

tentativas e motivo deixam o suporte responder "por que meu pedido foi cancelado?" sem abrir log.

PAGAMENTOS: AO RECEBER ESTOQUE.RESERVADO

```
async function processar({ pedidoId, correlationId, dados }) {
  // 1. idempotência: o mesmo evento não cobra duas vezes
  const { rows } = await pool.query(
    `INSERT INTO pagamentos (...) VALUES (..., 'PENDENTE')
     ON CONFLICT (pedido_id) DO NOTHING RETURNING id`, [...]);
  if (!rows.length) return;

  // 2. timeout + retry + circuit breaker
  const r = await cobrar({ valor, cartaoFinal, pedidoId });
  const status = r.aprovado ? 'APROVADO' : r.erro ? 'ERRO' : 'RECUSADO';
  await pool.query('UPDATE pagamentos SET status = $2, ...', ...);

  // 3. o resultado vira evento
  if (r.aprovado) await publicar('pagamentos', 'pagamento.aprovado', ...);
  else            await publicar('pagamentos', 'pagamento.recusado', ...);
}
```

Escuta o tópico estoque, não o pedidos: só cobra o que já tem estoque reservado.

Tudo que precisa vem no evento: valor, forma, cartão final, e-mail. Nenhuma consulta a outro serviço.

ERRO também publica recusado: o pedido não fica parado esperando um banco que caiu.

PAGAMENTOS: AO RECEBER PEDIDO.CANCELADO

```
async function aoCancelar({ pedidoId, correlationId, dados }) {
  // ainda não cobrei? ocupa o pedido_id com CANCELADO
  const { rows } = await pool.query(
    `INSERT INTO pagamentos (..., status) VALUES (..., 'CANCELADO')
     ON CONFLICT (pedido_id) DO NOTHING RETURNING id`, [...]);
  if (rows.length) return;   // a cobrança futura será ignorada

  // já existe: só estorna se foi APROVADO
  const pag = await buscar(pedidoId);
  if (pag.status !== 'APROVADO') return;
  await estornar(pag.codigo_transacao, pedidoId);
  await pool.query("UPDATE ... SET status = 'ESTORNADO'", ...);
  await publicar('pagamentos', 'pagamento.estornado', ...);
}
```

Pedidos: POST /pedidos/:id/cancelar em AGUARDANDO_ESTOQUE ou AGUARDANDO_PAGAMENTO publica pedido.cancelado.

Os dois casos da corrida (slide do cancelamento no meio da cobrança) num lugar só.

O pedido_id UNIQUE faz dois papéis: não cobrar duas vezes e não cobrar o que foi cancelado.

Estornar só APROVADO: RECUSADO, ERRO e ESTORNADO não têm nada a devolver. Evento repetido: nada acontece.

CATALOGO E PEDIDOS: REAGIR AO PAGAMENTO

```
// catalogo
'pagamento.aprovado': confirmar,  // reservado → vendido
'pagamento.recusado': liberar,    // COMPENSAÇÃO
'pedido.cancelado':   liberar,    // COMPENSAÇÃO

// liberar: só se a reserva ainda está 'reservada'
db.reservas.findOneAndUpdate(
  { pedidoId, status: 'reservada' },
  { $set: { status: 'liberada' } })
```

```
// pedidos
'pagamento.aprovado': e => mudarStatus(e.pedidoId,
    'PAGO', ['AGUARDANDO_PAGAMENTO']),
'pagamento.recusado': e => mudarStatus(e.pedidoId,
    'CANCELADO', ['AGUARDANDO_ESTOQUE',
                  'AGUARDANDO_PAGAMENTO'],
    e.dados.motivo),
```

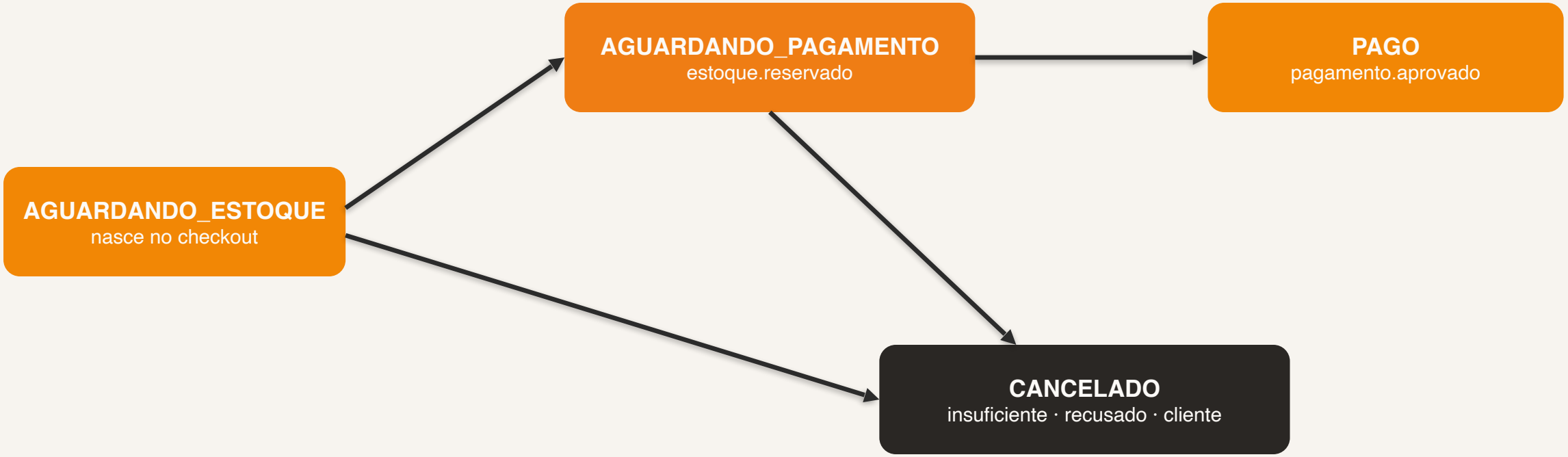
Confirmar: o produto sai do reservado. disponível já tinha diminuído na reserva.

Liberar: volta para o disponível. É a mesma função para recusa e cancelamento.

Os dois filtros de estado são a idempotência: liberar duas vezes não devolve duas vezes; PAGO não acontece num pedido já CANCELADO.

A referência já tem tudo isso: catalogo/src/estoque.js e o handlers de pedidos/src/index.js.

O PEDIDO, DO COMEÇO AO FIM



Três caminhos para CANCELADO: estoque.insuficiente (de AGUARDANDO_ESTOQUE), pagamento.recusado (de AGUARDANDO_ESTOQUE ou AGUARDANDO_PAGAMENTO) e o cliente cancelando.

PAGO é o fim feliz. No projeto, cancelar depois de pago não é permitido; o estorno acontece quando o pagamento termina depois de um cancelamento.

TESTANDO A SAGA

Cenário	Como provocar	Esperado
Caminho feliz	cartão 4111... ou pix	PAGO; estoque sai do reservado
Cartão recusado	cartão final 0002	CANCELADO; estoque volta
Banco instável	POST :9000/config { falhaPercent: 30 }	PAGO; tentativas 2 ou 3
Banco fora	{ falhaPercent: 100 }, várias compras	logs: CIRCUITO ABERTO; /health: aberto
Banco lento	{ latenciaMs: 3000 }	ERRO por timeout; pedido CANCELADO
Cancelar na cobrança	{ latenciaMs: 1500 }, checkout e cancelar	pagamento ESTORNADO ou CANCELADO

Nos arquivos .http: os três primeiros no fluxo-compra.http; os do banco-falso no demos.http, chamando localhost:9000/config direto.

PARA DISCUTIR EM DUPLA

1. O banco caiu e os pedidos foram cancelados. Como seria deixá-los esperando? O que muda nos status?
2. O estorno também falhou. Onde esse pedido fica registrado? Quem é avisado?
3. Por que reservar o estoque ANTES de cobrar, e não depois?
4. Pagamentos tem 3 réplicas. Cada uma tem o seu circuit breaker. Isso é um problema?
5. Onde vocês colocariam o retry: no pedidos chamando o catálogo, no pagamentos chamando o banco, ou nos dois?

RESPOSTA 1: ESPERAR O BANCO VOLTAR

1. *O banco caiu e os pedidos foram cancelados. Como seria deixá-los esperando? O que muda nos status?*

	Hoje: cancelar	Alternativa: esperar
Pagamento	ERRO	PENDENTE (ou AGUARDANDO_BANCO)
Pedido	CANCELADO	continua AGUARDANDO_PAGAMENTO: "em processamento"
Estoque	devolvido (compensação)	continua reservado

Quem tenta de novo: pausar o consumo com o circuito aberto (consumer.pause() no kafkajs): os eventos esperam no Kafka e são processados quando o circuito fecha. Ou uma rotina que, de tempos em tempos, cobra os PENDENTE.

Sempre com prazo: passadas, por exemplo, 2 horas, desiste: publica pagamento.recusado, a saga compensa e o cliente é avisado.

A troca: não perde a venda, mas prende o estoque, deixa o cliente sem resposta e dá mais trabalho. Decisão de negócio.

RESPOSTA 2: O ESTORNO FALHOU

2. *O estorno também falhou. Onde esse pedido fica registrado? Quem é avisado?*

Hoje, em lugar nenhum que preste: o erro vira uma linha de log e o pagamento continua APROVADO. O cliente pagou, o pedido está cancelado, e nenhuma tabela mostra isso.

01 **Gravar o estado:** status ESTORNO_PENDENTE na tabela de pagamentos. A informação fica no banco, não no log.

02 **Tentar de novo sozinho:** uma rotina reprocessa os pendentes, com espera crescente. O estorno pela referência é idempotente.

03 **Avisar:** alerta para o financeiro, evento num tópico de erros (DLQ), e-mail ao cliente: "estorno em processamento".

04 **Conciliar:** a rotina diária que compara o banco com o sistema pega o que escapou.

Regra: compensação que falha não pode sumir. Vira um estado gravado e um alerta.

RESPOSTA 3: RESERVAR ANTES DE COBRAR

3. *Por que reservar o estoque ANTES de cobrar, e não depois?*

DESFAZER UMA RESERVA	DESFAZER UMA COBRANÇA
Um UPDATE no nosso próprio banco.	Estorno no banco de outra empresa.
De graça, na hora.	Custa taxa e pode levar dias.
O cliente nem percebe.	O cliente vê a cobrança e o estorno no extrato.

Ordem pelo custo de desfazer: cobrando primeiro, todo pedido sem estoque viraria um estorno. Reservando primeiro, ele nem chega ao banco (passo 2 do roteiro).

O preço dessa escolha: o estoque fica preso enquanto o pagamento demora. Em produção, reservas têm validade: sem confirmação em X minutos, expiram e o estoque volta.

RESPOSTA 4: UM CIRCUITO POR RÉPLICA

4. *Pagamentos tem 3 réplicas. Cada uma tem o seu circuit breaker. Isso é um problema?*

réplica 1 ABERTO	**réplica 2** FECHADO, 3 falhas	**réplica 3** MEIO-ABERTO

Na maioria das vezes, não. O estado fica na memória de cada réplica:

- cada uma precisa das próprias 5 falhas: o banco recebe até 15 chamadas ruins antes de todas abrirem, e não 5;
- por alguns segundos elas discordam: uma aberta, outra ainda tentando.

O objetivo continua cumprido: cada réplica para de insistir e o banco recebe muito menos. Com o Kafka, cada uma já cuida só das suas partições.

Quando precisa de um estado só: guardar o circuito num lugar compartilhado (Redis) ou tirá-lo do código e pôr na infraestrutura (Envoy, Istio). Para o projeto, um por réplica está bom.

RESPOSTA 5: ONDE COLOCAR O RETRY

5. *Onde vocês colocariam o retry: no pedidos chamando o catálogo, no pagamentos chamando o banco, ou nos dois?*

Chamada	Retry?	Por quê
pagamentos → banco	sim, 3 tentativas	externo, falhas passageiras, ninguém esperando (é por evento), idempotente pela referência
pedidos → catálogo	no máximo 1, curta	o cliente está esperando o checkout: 3 tentativas estouram o timeout; melhor falhar rápido com 503
os dois, sem pensar	não	as tentativas se multiplicam a cada camada (slide da tempestade de retries)

Um detalhe: o Kong já faz retry. Todo service do Kong vem com retries: 5 para erros de conexão (se não consegue conectar, tenta de novo). É uma camada de retry que ninguém configurou.

Para requisições que não são idempotentes, como POST /pedidos, vale olhar esse número com cuidado.

O QUE ENTREGAR

1. **banco-falso** na porta 9000, fora do Kong: 0002 recusa, falhas e latência por /config.
2. **pagamentos** com Postgres próprio: escuta estoque.reservado, pedido_id UNIQUE, publica aprovado/recusado.
3. **Timeout, retry e circuit breaker** na chamada ao banco; estado do circuito no /health.
4. **Compensações:** catálogo devolve o estoque; pedidos vira CANCELADO com o motivo.
5. **Cancelamento:** pagamentos estorna o que foi cobrado ou bloqueia a cobrança futura.
6. **Notificações** de pagamento aprovado, recusado e estornado no Mailpit.
7. **Rotas /pagamentos no Kong** e os cenários da tabela nos arquivos .http.

CHECKPOINT: tag `aula-saga` no repositório. Os detalhes de cada item estão nos próximos 3 slides.

O QUE ENTREGAR 1/3: OS SERVIÇOS NOVOS

O quê	O que fazer	Pronto quando
banco-falso (serviço novo, sem banco de dados)	POST /cobrancas { valor, cartaoFinal, referencia } → 201 { aprovado, codigo: "TX-..." }. Cartão final 0002 → 402 { motivo: "saldo insuficiente" }. Antes de responder, espera latenciaMs; com falhaPercent % de chance, responde 503. POST /estornos { codigo, referencia } → 201. GET e POST /config { latenciaMs, falhaPercent }.	curl na porta 9000 com cartão 0002 devolve 402; depois de POST /config { "falhaPercent": 100 }, toda cobrança devolve 503.
pagamentos (serviço novo + pagamentos-db, Postgres)	Tabela pagamentos com pedido_id UNIQUE e os status PENDENTE, APROVADO, RECUSADO, ERRO, CANCELADO, ESTORNADO (slide da tabela). Consumer group pagamentos. Ao receber estoque.reservado: grava PENDENTE (ON CONFLICT DO NOTHING), cobra no banco, atualiza o status, a quantidade de tentativas e o motivo, e publica no tópico pagamentos: pagamento.aprovado { usuarioId, clienteEmail, valor, codigoTransacao } ou pagamento.recusado { ..., motivo }.	Uma compra com pix gera uma linha APROVADO com codigo_transacao. O mesmo evento entregue de novo não cria uma segunda linha nem cobra de novo.
rotas de pagamentos	GET /pagamentos/:pedidoId: só o dono do pedido ou admin (404 se não existir). GET /pagamentos: só admin. GET /health: { banco, circuitoBancoExterno: fechado | aberto | meio-aberto }.	Pelo Kong: o cliente vê o próprio pagamento; outro cliente recebe 404; GET /pagamentos com perfil cliente recebe 403.

O QUE ENTREGAR 2/3: RESILIÊNCIA E COMPENSAÇÕES

O quê	O que fazer	Pronto quando
chamada ao banco (em pagamentos)	Timeout de 2 s em toda chamada. Retry: até 3 tentativas, esperando 500 ms e 1 s, só para timeout, erro de rede e 5xx (402 não repete). Circuit breaker (opossum ou o seu): abre com muitas falhas, espera 15 s, testa no meio-aberto. Esgotou as tentativas ou circuito aberto: status ERRO e pagamento.recusado com motivo "banco indisponível".	falhaPercent 30: os pedidos saem PAGO, alguns com 2 ou 3 tentativas. falhaPercent 100: log CIRCUITO ABERTO e /health "aberto"; volta a "fechado" sozinho.
catalogo	pagamento.aprovado: confirma a reserva (reservado diminui). pagamento.recusado e pedido.cancelado: devolve o estoque (disponivel aumenta, reservado diminui). As duas só agem se a reserva ainda estiver reservada: evento repetido não faz nada.	Compra com cartão 0002: o disponivel do produto volta ao valor de antes da compra.
pedidos	Escuta o tópico pagamentos: aprovado → PAGO (só de AGUARDANDO_PAGAMENTO); recusado → CANCELADO com o motivo (de AGUARDANDO_ESTOQUE ou AGUARDANDO_PAGAMENTO). POST /pedidos/:id/cancelar (dono): só em AGUARDANDO_ESTOQUE ou AGUARDANDO_PAGAMENTO, senão 409; muda para CANCELADO e publica pedido.cancelado.	GET /pedidos/:id mostra PAGO, ou CANCELADO com o motivo. Cancelar um pedido PAGO devolve 409.
pagamentos, ao cancelar	Ao receber pedido.cancelado: grava CANCELADO (ON CONFLICT DO NOTHING). Se já existia e está APROVADO: POST /estornos, status ESTORNADO e publica pagamento.estornado.	latenciaMs 1500, checkout e cancelar em seguida: pagamento ESTORNADO (ou CANCELADO) e o estoque volta.
notificacoes	Escuta o tópico pagamentos: e-mail de pagamento aprovado, recusado (com o motivo) e estornado.	Os três e-mails aparecem no Mailpit (localhost:8025).

O QUE ENTREGAR 3/3: COMPOSE, KONG E TESTES

- ✓ **docker-compose.yml:** pagamentos, pagamentos-db e banco-falso. Pagamentos sem ports e esperando o Kafka (healthy); banco-falso com "9000:3000".
- ✓ **kong.yml:** rota /pagamentos para o serviço pagamentos, com jwt (qualquer perfil logado; o serviço confere dono e admin) e /health/pagamentos público.
- ✓ **http/fluxo-compra.http:** compra aprovada, compra com cartão 0002 e compra cancelada com estorno, cada uma consultando o pedido e o pagamento no fim.
- ✓ **http/demos.http:** POST localhost:9000/config para falhaPercent 30, falhaPercent 100 (circuito abre) e latenciaMs 3000 (timeout), e a volta ao normal.
- ✓ **README:** tabela de eventos com os quatro novos (pagamento.aprovado, recusado, estornado e pedido.cancelado) e como demonstrar o circuit breaker.
- ✓ **Logs:** cada tentativa de cobrança, a abertura e o fechamento do circuito e cada compensação, sempre com o pedidoId e o correlationId.

Pronto quando os arquivos .http rodam do começo ao fim com o compose recém-criado (docker compose down -v && docker compose up -d --build). Tag: `aula-saga`.

PRÓXIMA AULA

01 Cache: Redis na vitrine do catálogo, e o que fazer quando o produto muda.

02 Observabilidade: logs com correlationId, healthchecks e o que monitorar.

03 O lag do Kafka e o estado do circuito como sinais de saúde.

04 Preparando o terreno para o deploy no servidor.

Contato



Augusto Gehrke

https://www.linkedin.com/in/augusto-g-b54a5bb7

augusto.gehrke@amf.edu.br

augustogehrke@outlook.com